

Operating Systems (OS)

Complete Interview Q&A Guide — Basic to Advanced

Covers every OS concept from **Fundamentals, Process Management, Threads, CPU Scheduling, Synchronization, Deadlocks, Memory Management, Virtual Memory, File Systems, I/O Systems, Security**, and more.

☐ Table of Contents

1. [OS Fundamentals](#)
 2. [Process Management](#)
 3. [Threads & Concurrency](#)
 4. [CPU Scheduling](#)
 5. [Process Synchronization](#)
 6. [Deadlocks](#)
 7. [Memory Management](#)
 8. [Virtual Memory](#)
 9. [File Systems](#)
 10. [I/O Systems & Device Management](#)
 11. [Storage Management](#)
 12. [Security & Protection](#)
 13. [Virtualization & Containers](#)
 14. [Distributed Operating Systems](#)
 15. [Linux Internals & System Calls](#)
 16. [Advanced OS Concepts](#)
-

1. OS Fundamentals

Q1. What is an Operating System? What are its goals?

A: An Operating System (OS) is **system software** that acts as an intermediary between users/applications and computer hardware. It manages hardware resources and provides a convenient environment for program execution.

Goals of an OS:

- **Convenience:** Make the computer easy to use (abstract hardware complexity).
- **Efficiency:** Use hardware resources (CPU, memory, I/O) efficiently.
- **Ability to evolve:** Allow new system functions to be developed without disrupting existing services.

Two views of an OS:

- **Resource Manager:** Manages CPU, memory, disk, I/O devices — allocates, schedules, and protects resources among competing processes.
- **Extended Machine / Virtual Machine:** Provides a higher-level abstraction of hardware, hiding low-level details from applications.

Q2. What are the functions of an Operating System?

A:

- **Process Management:** Create, schedule, terminate processes and threads; handle synchronization and inter-process communication.
- **Memory Management:** Allocate and deallocate memory; manage virtual memory, paging, and segmentation.
- **File System Management:** Create, delete, read, write files and directories; manage storage space.
- **I/O Device Management:** Manage device drivers, buffering, caching, and spooling.
- **Security and Protection:** Authenticate users; enforce access control; protect processes from each other.
- **Networking:** Provide network protocol stack, socket APIs.
- **User Interface:** CLI (shell) or GUI for user interaction.
- **Error Detection and Handling:** Detect and recover from hardware/software errors.

Q3. What are the types of Operating Systems?

A:

Type	Description	Examples
Batch OS	Jobs collected in batches; executed without user interaction	IBM OS/360
Multiprogramming OS	Multiple programs in memory; CPU switches when I/O wait	Early UNIX
Multitasking/Time-sharing OS	CPU rapidly switches between users/processes; interactive	Linux, Windows
Real-Time OS (RTOS)	Strict timing deadlines;	FreeRTOS, VxWorks

Type	Description	Examples
	predictable response time	
Distributed OS	Multiple computers appear as single system	Amoeba, Plan 9
Network OS	Networking services; each machine still autonomous	Novell NetWare
Embedded OS	Designed for embedded devices; resource-constrained	Android, iOS, µC/OS
Mobile OS	Touch-optimized for smartphones/tablets	Android, iOS
Cloud OS	Manage cloud infrastructure	OpenStack

Q4. What is Kernel? Types of Kernels?

A: The **kernel** is the **core of the OS** — the part that runs with full hardware privileges (kernel mode) and directly manages hardware resources. It's always resident in memory.

Types of Kernels:

Monolithic Kernel:

- Entire OS (file system, drivers, memory management, scheduler) runs in a single large kernel space.
- Fast (no context switch for kernel services), but large and hard to maintain.
- A bug in any module can crash the entire system.
- Examples: Linux, early UNIX.

Microkernel:

- Only minimal services run in kernel space (IPC, basic scheduling, memory management).
- All other services (file system, device drivers, networking) run in user space as servers.
- More modular, stable (server crash doesn't kill kernel), but slower (message passing overhead).
- Examples: Mach, QNX, MINIX 3, L4.

Hybrid Kernel:

- Compromise: some services in kernel space (performance-critical), some in user space.
- Examples: Windows NT, macOS (XNU = Mach microkernel + BSD monolithic components).

Exokernel:

- Extremely minimal kernel — only multiplexes hardware. All abstractions pushed to library OS (libOS).

- Research systems: MIT Exokernel.

Nanokernel / Unikernel:

- Even smaller than microkernel; application compiled with only the OS code it needs.
- Examples: MirageOS, IncludeOS. Used in cloud/containers.

Q5. What is the difference between kernel mode and user mode?

A:

Aspect	Kernel Mode (Privileged)	User Mode (Unprivileged)
Access	Full hardware access	Restricted; cannot directly access hardware
Privileges	Execute any instruction, access any memory	Limited instruction set; own memory only
CPU ring	Ring 0 (x86)	Ring 3 (x86)
Crash impact	Kernel crash = system crash	User process crash is isolated
Runs	OS kernel, device drivers	User applications, libraries
Mode switch	System call, interrupt, exception	Return from system call

The **dual-mode** operation is fundamental to OS protection — user programs cannot directly access hardware or other processes' memory.

Q6. What is a system call?

A: A system call is the **programmatic interface** through which a user-space program requests a service from the kernel. It is the only safe, controlled way for user programs to cross the user-mode/kernel-mode boundary.

Mechanism:

1. User program calls a library function (e.g., `read( )` in C).
2. Library sets up arguments in registers, executes a **trap instruction** (`INT 0x80`, `SYSCALL`).
3. CPU switches to kernel mode, jumps to the OS system call handler.
4. Kernel validates parameters, performs the requested service.
5. Kernel places result in register, executes `SYSRET`, CPU returns to user mode.
6. Library function returns result to the caller.

Categories of System Calls:

Category	Examples
Process control	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code> , <code>getpid()</code>
File management	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code> , <code>lseek()</code>
Device management	<code>ioctl()</code> , <code>read()</code> , <code>write()</code> on device files
Information	<code>getpid()</code> , <code>gettimeofday()</code> , <code>uname()</code>
Communication	<code>pipe()</code> , <code>socket()</code> , <code>send()</code> , <code>recv()</code> , <code>mmap()</code>
Memory	<code>brk()</code> , <code>mmap()</code> , <code>munmap()</code>
Security	<code>chmod()</code> , <code>chown()</code> , <code>setuid()</code>

Q7. What is the difference between a program, a process, and a thread?

A:

- **Program:** A **passive** entity — a file on disk containing executable code (binary instructions + data). Static.
- **Process:** An **active** instance of a program in execution. Has its own memory space (code, data, heap, stack), file descriptors, and OS resources. Dynamic.
- **Thread:** A **lightweight unit of execution within a process**. Shares the process's memory space (code, data, heap) but has its own stack and registers.

```

Program (disk)
  ↓ loaded by OS (exec)
Process (memory)
  ├── Code Segment
  ├── Data Segment (global vars)
  ├── Heap (dynamic allocation)
  └── Stack(s) → one per thread
      Thread 1 (main)
      Thread 2
      Thread N

```

Q8. What is an OS shell? Difference between CLI and GUI?

A: A shell is a **user interface** to the OS — it accepts commands and translates them into system calls.

- **CLI (Command Line Interface):** Text-based. User types commands. Examples: `bash`, `zsh`, `PowerShell`. Faster for scripting, automation, remote access.
- **GUI (Graphical User Interface):** Visual, mouse-driven. `Windows Explorer`, `macOS Finder`. Easier for non-technical users.

Popular Linux shells: `bash` (Bourne Again Shell), `zsh`, `fish`, `sh`. Shell scripting automates tasks

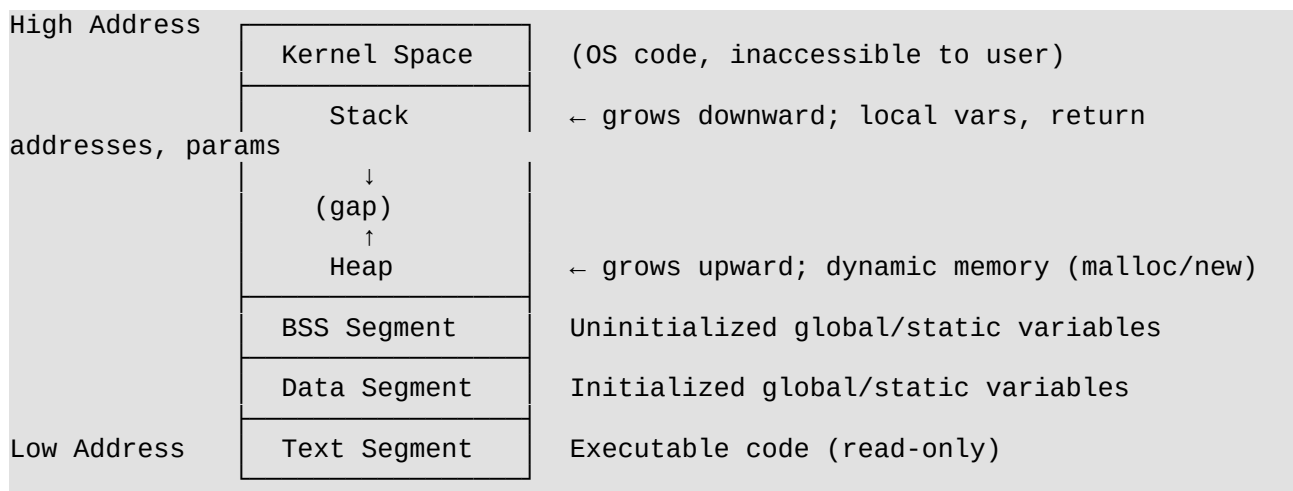
using OS commands.

2. Process Management

Q9. What is a Process? What does its memory layout look like?

A: A process is a **program in execution** — an active entity with resources (memory, CPU time, open files, I/O devices).

Process Memory Layout (bottom to top in virtual address space):



Q10. What is a Process Control Block (PCB)?

A: A PCB (also called Task Control Block) is a **kernel data structure** that stores all information about a process. It's the OS's representation of a process.

PCB contents:

Field	Description
Process ID (PID)	Unique identifier
Process State	Running, Ready, Blocked, etc.
Program Counter (PC)	Address of next instruction
CPU Registers	All register values (saved on context switch)
CPU Scheduling Info	Priority, scheduling queue pointers
Memory Management Info	Base/limit registers, page tables
Accounting Info	CPU time used, wall clock time, job number
I/O Status Info	Open file list, allocated devices
Parent PID (PPID)	Parent process identifier
Signals	Pending signal mask

When a context switch occurs, the CPU state of the running process is saved into its PCB, and the PCB of the next process is loaded.

Q11. What are the states of a process?

A:

Five-State Model:



- **New:** Process being created (PCB allocated, not yet in ready queue).
- **Ready:** In memory, waiting for CPU. In the ready queue.
- **Running:** Currently executing on CPU. (Only one per CPU core at a time.)
- **Blocked/Waiting:** Waiting for I/O completion, event, or resource. Not eligible for CPU.
- **Terminated:** Process finished execution; PCB being cleaned up.

Additional states (with virtual memory/swapping):

- **Suspended Ready:** Swapped out of memory but ready to run when swapped back.
 - **Suspended Blocked:** Swapped out and still waiting for an event.
-

Q12. What is a context switch?

A: A context switch is the process of **saving the state of the currently running process** and **restoring the state of the next process** to be run.

Steps:

1. Interrupt or system call triggers kernel.
2. Save current process's CPU registers, PC, stack pointer into its PCB.
3. Update process state (Running → Ready or Blocked).
4. Run scheduler to select next process.
5. Load next process's state from its PCB.
6. Update memory management registers (switch page tables).
7. Resume execution of next process.

Cost: Context switching is pure overhead — no useful work is done during the switch. Modern CPUs minimize this with fast register save/restore, but TLB flushes on context switch are expensive

(mitigated by ASIDs).

Q13. What is `fork()`, `exec()`, and `wait()` in Unix/Linux?

A:

fork(): Creates a new process by duplicating the calling process.

- Parent gets child's PID as return value; child gets 0.
- Child is an exact copy (PCB, memory, file descriptors) — **copy-on-write** optimization.

exec(): Replaces the current process's memory image with a new program.

- The process keeps its PID but loads new code.
- Variants: `execl()`, `execv()`, `execve()`, etc.

wait(): Parent blocks until a child process terminates; collects exit status.

```
pid_t pid = fork();
if (pid == 0) {
    // Child process
    execl("/bin/ls", "ls", "-l", NULL); // Replace with 'ls' command
} else if (pid > 0) {
    // Parent process
    wait(NULL); // Wait for child to finish
    printf("Child done\n");
} else {
    perror("fork failed");
}
```

Q14. What is a zombie process and an orphan process?

A:

Zombie Process:

- A process that has **finished execution** (`exit()` called) but its PCB still exists because its parent has NOT yet called `wait()` to collect its exit status.
- It holds no resources (memory freed) but occupies a PID and entry in the process table.
- Becomes a problem if the parent never calls `wait()` — zombie accumulates, exhausting PID space.
- Fix: Parent calls `wait()` / `waitpid()`; or install a `SIGCHLD` handler.

Orphan Process:

- A process whose **parent has terminated** while the child is still running.
 - Adopted by the **init process (PID 1)** which periodically calls `wait()`.
 - Not harmful by itself.
-

Q15. What is Inter-Process Communication (IPC)?

A: IPC mechanisms allow processes to **communicate and synchronize** with each other.

IPC Mechanism	Description	Direction	Speed
Pipe	Unidirectional byte stream between related processes (parent-child)	Unidirectional	Fast
Named Pipe (FIFO)	Like pipe but unrelated processes can use it; exists as file	Unidirectional	Fast
Message Queue	Messages sent to a queue; received by any process with access	Bidirectional	Moderate
Shared Memory	Fastest IPC — processes map same physical memory; requires synchronization	Bidirectional	Fastest
Semaphore	Signaling mechanism for synchronization; not for data transfer	—	Fast
Signal	Asynchronous notification (SIGKILL, SIGTERM, SIGSEGV)	Unidirectional	Fast
Socket	Network or Unix domain sockets; works across machines	Bidirectional	Variable
Memory-mapped File	File mapped into virtual address space; shared between processes	Bidirectional	Fast

Shared memory is the fastest IPC (no kernel involvement after setup) but requires explicit synchronization (semaphores, mutexes).

3. Threads & Concurrency

Q16. What is a thread? Why use threads?

A: A thread is the **smallest unit of CPU execution** within a process. All threads of a process share:

- Code segment, data segment, heap, open files, signal handlers.

Each thread has its own:

- Stack (local variables, function call frames)
- Registers (including Program Counter)
- Thread ID
- Signal mask

Why threads?:

- **Responsiveness:** UI thread stays responsive while background thread processes data.
- **Resource sharing:** Threads share memory — cheaper communication than IPC.
- **Economy:** Creating a thread is much cheaper than creating a process (no address space duplication).
- **Scalability:** Exploit multicore CPUs — multiple threads truly run in parallel.

Q17. What are the differences between a process and a thread?

A:

Aspect	Process	Thread
Address space	Own separate address space	Shared with other threads in same process
Creation cost	Heavy (fork + copy address space)	Light (just stack + registers)
Communication	Requires IPC	Direct shared memory access
Context switch	Expensive (TLB flush, page table switch)	Cheaper (same address space)
Isolation	Crash doesn't affect others	Crash can kill entire process
Synchronization	Less needed (separate memory)	Required (shared memory → race conditions)
OS resources	Own file descriptors, signals, etc.	Shared within process

Q18. What are user-level threads vs kernel-level threads?

A:

User-Level Threads (ULT):

- Managed by a user-space **thread library** (e.g., POSIX green threads).
- Kernel is unaware — sees only one process.
- **Pros:** Fast creation and switching (no kernel call), portable.
- **Cons:** If one thread blocks on I/O, the entire process blocks (kernel blocks the process). Cannot exploit multiple CPUs.

Kernel-Level Threads (KLT):

- Managed directly by the OS kernel.
- Kernel schedules each thread independently on CPUs.
- **Pros:** True parallelism on multicore; one blocking thread doesn't block others.
- **Cons:** Thread creation/switching requires kernel call — higher overhead.
- Examples: POSIX pthreads on Linux (mapped 1:1 to kernel threads).

Threading Models:

- **Many-to-One (M:1):** Many ULTs → one KLT. No parallelism.
- **One-to-One (1:1):** Each ULT → one KLT. True parallelism; high overhead. Linux pthreads.
- **Many-to-Many (M:N):** M ULTs → N KLTs ($N \leq M$). Best of both worlds. Complex. Solaris, Windows fibers.
- **Two-level (hybrid):** Like M:N but also allows 1:1 binding for critical threads.

Q19. What is a thread pool?

A: A thread pool is a collection of **pre-created worker threads** that wait for tasks and execute them, rather than creating and destroying threads for each task.

Benefits:

- Eliminates thread creation overhead per task.
- Limits maximum thread count (prevents resource exhaustion).
- Improves response time (threads ready immediately).
- Better CPU utilization with proper pool sizing.

```
Task Queue → [ Thread 1 ]
              → [ Thread 2 ]   worker threads pick tasks and execute
              → [ Thread 3 ]
```

Used everywhere: Web servers (Tomcat, Nginx workers), Java `ExecutorService`, Python `ThreadPoolExecutor`, database connection pools.

Sizing rule of thumb: I/O-bound: more threads (e.g., $2 \times$ CPU cores); CPU-bound: $\approx$ CPU core count.

Q20. What is the difference between concurrency and parallelism?

A:

- **Concurrency:** Multiple tasks are **in progress at the same time** — but not necessarily executing simultaneously. The CPU interleaves execution. Achieved even on a single core (time-slicing). About dealing with multiple things at once.

- **Parallelism:** Multiple tasks are **literally executing simultaneously** on multiple CPU cores.
A subset of concurrency. About doing multiple things at once.

Concurrency (1 core): T1— T2— T1— T2— (interleaved)
Parallelism (2 cores): Core1: T1—
Core2: T2— (simultaneous)

A concurrent program may or may not be parallel; a parallel program is always concurrent.

4. CPU Scheduling

Q21. What is CPU scheduling? Why is it needed?

A: CPU scheduling is the process of **deciding which process/thread in the ready queue gets to run on the CPU** and for how long. It's needed because:

- Multiple processes compete for CPU time.
- CPU should be kept as busy as possible (maximize utilization).
- Different processes have different priorities and urgency.
- Multiprogramming requires efficient CPU multiplexing.

Types of scheduling:

- **Long-term (Job) Scheduler:** Selects which jobs are admitted into the system (from job pool to ready queue). Controls degree of multiprogramming.
- **Short-term (CPU) Scheduler:** Selects which ready process gets the CPU next. Runs very frequently (milliseconds).
- **Medium-term Scheduler:** Manages swapping — moves processes between memory and disk to control multiprogramming degree.

Q22. What are CPU scheduling criteria?

A:

Criterion	Goal	Maximize/Minimize
CPU Utilization	Keep CPU busy	Maximize
Throughput	Jobs completed per unit time	Maximize
Turnaround Time	Total time from submission to completion	Minimize
Waiting Time	Total time spent in ready queue	Minimize
Response Time	Time from request to first response	Minimize
Fairness	Each process gets fair CPU share	Balanced

- **Turnaround Time** = Completion Time – Arrival Time
 - **Waiting Time** = Turnaround Time – Burst Time
 - **Response Time** = First CPU execution – Arrival Time
-

Q23. What is preemptive vs non-preemptive scheduling?

A:

- **Non-preemptive (Cooperative):** Once a process gets the CPU, it runs until it voluntarily relinquishes (I/O wait, termination, or explicit yield). Simple but can cause starvation of high-priority processes.
 - **Preemptive:** OS can forcibly take CPU from a running process (timer interrupt, higher-priority process arrives). Enables time-sharing and real-time responsiveness. Risk of race conditions in shared data.
-

Q24. Explain all CPU scheduling algorithms.

A:

1. First-Come, First-Served (FCFS):

- Processes served in arrival order.
- Non-preemptive.
- **Problem:** Convoy effect — short jobs stuck behind long jobs.
- Average waiting time can be high.

Process: P1(24ms), P2(3ms), P3(3ms) (all arrive at t=0)

Order: P1 → P2 → P3

WT: P1=0, P2=24, P3=27 → Avg WT = 17ms

If order reversed: P2 → P3 → P1

WT: P2=0, P3=3, P1=6 → Avg WT = 3ms

2. Shortest Job First (SJF):

- Schedule process with the **shortest next CPU burst** first.
- **Optimal average waiting time** (among non-preemptive algorithms).
- **Problem:** Starvation of long jobs; requires knowing next burst length (impossible in practice → use exponential averaging prediction).
- Non-preemptive version; preemptive = SRTF.

Exponential averaging: $\tau(n+1) = \alpha \cdot t(n) + (1-\alpha) \cdot \tau(n)$ where $t(n)$ = actual burst,

$\tau(n)$ = predicted.

3. Shortest Remaining Time First (SRTF):

- Preemptive version of SJF.
 - When a new process arrives with a shorter remaining time than the current process, preempt.
 - Optimal average waiting time overall, but still requires burst prediction and causes starvation.
-

4. Round Robin (RR):

- Each process gets a fixed **time quantum (q)** in turn.
- After quantum expires, process is preempted and added to end of ready queue.
- **Fair:** Every process gets CPU within $(n-1) \times q$ time (n = number of processes).
- **No starvation.**
- **Time quantum choice:** Too small $\rightarrow$ too many context switches (overhead). Too large $\rightarrow$ degenerates to FCFS. Rule of thumb: $q > 80\%$ of CPU bursts.

Processes: P1(24ms), P2(3ms), P3(3ms), quantum=4ms
Order: P1(4) $\rightarrow$ P2(3) $\rightarrow$ P3(3) $\rightarrow$ P1(4) $\rightarrow$ P1(4) $\rightarrow$ P1(4) $\rightarrow$ P1(4) $\rightarrow$ P1(4)

5. Priority Scheduling:

- Each process has a priority number; highest priority (often lowest number) runs first.
 - Can be preemptive or non-preemptive.
 - **Problem:** Starvation — low-priority processes may never run.
 - **Solution: Aging** — gradually increase priority of waiting processes over time.
 - SJF is priority scheduling where $\text{priority} = 1/\text{burst_time}$.
-

6. Multilevel Queue Scheduling:

- Ready queue divided into **multiple queues** based on process type (e.g., foreground/interactive, background/batch, system).
 - Each queue has its own scheduling algorithm.
 - **Fixed priority** between queues (foreground always before background) $\rightarrow$ possible starvation of lower queues.
-

7. Multilevel Feedback Queue (MLFQ):

- Like multilevel queue, but **processes can move between queues** based on behavior.

- New processes start in highest-priority queue (short quantum).
- If they use full quantum without blocking → demoted to lower-priority queue (longer quantum).
- If they block early (I/O-bound) → stay in or promoted to higher queue.
- **Most flexible and widely used** (basis for Linux CFS, Windows scheduler).
- Aging: boost priority of starved processes.

8. Completely Fair Scheduler (CFS) — Linux:

- Linux's default scheduler (for normal processes).
- Uses a **red-black tree** ordered by **vruntime** (virtual runtime — how much CPU time a process has used, normalized by weight/priority).
- Always picks the process with the **smallest vruntime** (least CPU time so far) → fairness.
- Nice value (-20 to +19) maps to weight; higher priority → lower weight penalty → accumulates vruntime slower → runs more.

Q25. What is the difference between preemptive and non-preemptive versions of SJF?

A:

Aspect	SJF (Non-preemptive)	SRTF (Preemptive)
Preemption	No — runs to completion	Yes — preempted if shorter job arrives
Optimality	Optimal for non-preemptive	Optimal overall (minimum avg waiting time)
Starvation	Yes	Yes (more severe)
Overhead	Low	Higher (more context switches)

Q26. What is the difference between dispatcher and scheduler?

A:

- **Scheduler:** Decides **which process** to run next (policy decision). Runs less frequently.
- **Dispatcher:** Actually **gives control of the CPU** to the selected process (mechanism). Runs on every context switch.

Dispatcher tasks: Context switch (save/restore), switch to user mode, jump to correct PC location.

Dispatch latency: Time for dispatcher to stop one process and start another. Should be minimized.

5. Process Synchronization

Q27. What is the critical section problem?

A: When multiple processes/threads access **shared data concurrently**, and at least one modifies it, the result depends on the order of access — called a **race condition**.

A **critical section** is a code segment where shared data is accessed.

Requirements for a valid critical section solution:

1. **Mutual Exclusion:** Only one process can be in its critical section at a time.
 2. **Progress:** If no process is in the critical section and some want to enter, selection of the next one cannot be postponed indefinitely.
 3. **Bounded Waiting:** There exists a bound on the number of times other processes can enter before a waiting process is granted access (no starvation).
-

Q28. What is Peterson's Solution?

A: Peterson's algorithm is a classic **software-based** solution for mutual exclusion between two processes.

```
// Shared variables
int turn;           // Whose turn it is
bool flag[2];      // flag[i] = true: process i wants to enter CS

// Process i (j = 1-i):
flag[i] = true;
turn = j;           // Give turn to the other
while (flag[j] && turn == j); // Busy wait
// — CRITICAL SECTION —
flag[i] = false;
// — REMAINDER SECTION —
```

Satisfies all three requirements. **Limitation:** Only works for two processes; relies on correct memory ordering (may fail on modern CPUs with reordering — needs memory barriers).

Q29. What are hardware synchronization primitives?

A: Modern CPUs provide atomic hardware instructions for synchronization:

Test-and-Set (TAS):

```
bool TestAndSet(bool *target) {
    bool rv = *target;
    *target = true; // Atomically set to true
    return rv;      // Return old value
}

// Spinlock using TAS:
while (TestAndSet(&lock)); // Spin until lock was false
// Critical section
lock = false;
```


Compare-and-Swap (CAS) — more powerful:

```
int CompareAndSwap(int *value, int expected, int new_val) {
    int temp = *value;
    if (*value == expected) *value = new_val;
    return temp;
}
// Used in lock-free data structures
```

Fetch-and-Add: Atomically adds to a value and returns old value. Used for ticket locks.

Memory Barrier (Fence): Ensures all memory operations before the barrier are visible to other cores before operations after the barrier. Critical on CPUs with weak memory models (ARM, POWER).

Q30. What is a Mutex?

A: A **Mutex (Mutual Exclusion lock)** is a synchronization primitive that ensures only one thread enters the critical section at a time.

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

// Thread A and Thread B:
pthread_mutex_lock(&lock);      // Acquire – blocks if held
// — Critical Section —
pthread_mutex_unlock(&lock);    // Release
```

Properties:

- **Ownership:** Only the thread that locked the mutex can unlock it.
- **Blocking:** A thread trying to acquire a locked mutex is blocked (put to sleep) — not spinning.
- **Non-recursive** (typically): A thread that holds a mutex and tries to lock it again will deadlock.

Mutex vs Spinlock:

- **Mutex:** Thread sleeps while waiting → good for long critical sections; context switch overhead.
 - **Spinlock:** Thread busy-waits (spins) → good for very short critical sections on multicore; wastes CPU if held long.
-

Q31. What is a Semaphore?

A: A semaphore is an integer variable S supporting two atomic operations:

- **wait(S) / P(S) / down(S):** Decrement S ; if $S < 0$, block the calling process.
- **signal(S) / V(S) / up(S):** Increment S ; if processes are blocked ($S \leq 0$), wake one up.

```
// Binary Semaphore (Mutex):
sem_t mutex;
sem_init(&mutex, 0, 1); // Initial value = 1
```

```
sem_wait(&mutex);    // P – lock
// Critical Section
sem_post(&mutex);    // V – unlock

// Counting Semaphore (resource pool of N):
sem_init(&resources, 0, N); // N available
sem_wait(&resources);    // Acquire one resource
// Use resource
sem_post(&resources);    // Release resource
```

Types:

- **Binary Semaphore** (0 or 1): Works like a mutex (but without ownership).
- **Counting Semaphore**: Can be any non-negative value; controls access to a resource pool.

Key difference from mutex: Semaphore has no ownership — any thread can signal it. Used for signaling between threads.

Q32. What is a Monitor?

A: A monitor is a **high-level synchronization construct** that combines:

- A set of shared variables (mutual exclusion guaranteed by the monitor)
- Procedures/methods to operate on those variables
- **Condition variables** for waiting on specific conditions

Only one process/thread can be active inside a monitor at a time (implicit mutex).

```
// Java synchronized methods = monitor
class BoundedBuffer {
    synchronized void produce(int item) {
        while (buffer.isFull()) wait(); // Condition wait
        buffer.add(item);
        notifyAll(); // Signal waiting consumers
    }

    synchronized int consume() {
        while (buffer.isEmpty()) wait();
        int item = buffer.remove();
        notifyAll(); // Signal waiting producers
        return item;
    }
}
```

Monitors are safer than semaphores (mutual exclusion is automatic, not programmer's responsibility).

Q33. What is the Producer-Consumer Problem?

A: Classic synchronization problem: A producer generates data and puts it in a bounded buffer; a consumer takes data from the buffer. Must synchronize to:

- Prevent producer from adding to a full buffer.

- Prevent consumer from removing from an empty buffer.
- Ensure mutual exclusion on buffer access.

```
sem_t mutex, empty, full;
sem_init(&mutex, 0, 1);    // Mutual exclusion
sem_init(&empty, 0, N);    // N empty slots initially
sem_init(&full, 0, 0);     // 0 full slots initially

// Producer:
sem_wait(&empty);    // Wait for empty slot
sem_wait(&mutex);
// Add item to buffer
sem_post(&mutex);
sem_post(&full);     // Signal one more full slot

// Consumer:
sem_wait(&full);     // Wait for full slot
sem_wait(&mutex);
// Remove item from buffer
sem_post(&mutex);
sem_post(&empty);    // Signal one more empty slot
```

Q34. What is the Readers-Writers Problem?

A: Multiple readers can read shared data simultaneously (no conflict), but a writer needs exclusive access.

First Readers-Writers Problem (Readers preference — no reader waits unless a writer holds access):

- New readers can jump ahead of waiting writers → writer starvation.

Second Readers-Writers Problem (Writers preference):

- Once a writer is ready, no new readers start → reader starvation.

Solution sketch:

```
// Shared: read_count, mutex (protect read_count), rw_lock (exclusive write)
// Reader:
sem_wait(&mutex);
read_count++;
if (read_count == 1) sem_wait(&rw_lock); // First reader locks writers out
sem_post(&mutex);
// — Read data —
sem_wait(&mutex);
read_count--;
if (read_count == 0) sem_post(&rw_lock); // Last reader unlocks writers
sem_post(&mutex);

// Writer:
sem_wait(&rw_lock); // Exclusive access
// — Write data —
sem_post(&rw_lock);
```

Q35. What is the Dining Philosophers Problem?

A: Five philosophers sit around a table with 5 forks (one between each pair). A philosopher either thinks or eats. To eat, they need BOTH adjacent forks. They pick up one fork at a time.

Deadlock scenario: All 5 pick up their left fork simultaneously → all wait for the right fork → deadlock.

Solutions:

1. **Allow at most 4 philosophers** to sit at a time (resource limitation).
 2. **Asymmetric solution:** Odd-numbered philosophers pick left then right; even pick right then left.
 3. **Chandy/Misra solution:** Forks can be "dirty" or "clean"; philosophers send requests.
 4. **Pick up both or none:** Use a mutex to ensure both forks are grabbed atomically.
-

Q36. What is a spinlock vs a blocking lock?

A:

Spinlock:

- Thread **busy-waits** (loops checking) until the lock is free.
- Wastes CPU cycles while waiting.
- Appropriate when: lock is held for a very short time, cost of sleeping > spinning time, on multicore (thread waiting on one core, holder on another).
- Used in OS kernel for short critical sections.

Blocking Lock (Mutex):

- Thread is **descheduled** (put to sleep) when it can't acquire.
 - Kernel wakes it when lock is released (context switch overhead).
 - Appropriate when: lock may be held for a long time, on single-core, or when spinning would starve others.
-

6. Deadlocks

Q37. What is a deadlock? What are its necessary conditions?

A: A deadlock is a state where a **set of processes are blocked forever**, each waiting for a resource held by another process in the set.

Coffman's Four Necessary Conditions (ALL must hold simultaneously):

1. **Mutual Exclusion:** Resources are non-shareable — only one process can use a resource at a time.
2. **Hold and Wait:** A process holds at least one resource while waiting to acquire additional

resources held by others.

3. **No Preemption:** Resources cannot be forcibly taken from a process; they must be released voluntarily.
4. **Circular Wait:** A circular chain of processes exists where each waits for a resource held by the next: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$.

Q38. What is a Resource Allocation Graph (RAG)?

A: A RAG is a directed graph used to represent the state of resource allocation:

- **Process node:** Circle ($P1, P2 \dots$)
- **Resource node:** Rectangle ($R1, R2 \dots$) with dots for instances
- **Assignment edge:** $R \rightarrow P$ (resource assigned to process)
- **Request edge:** $P \rightarrow R$ (process requesting resource)

Deadlock detection:

- **Single instance** per resource: Deadlock $\iff$ cycle in RAG.
- **Multiple instances:** Cycle is necessary but not sufficient. Use Banker's Algorithm for detection.

Q39. What are the strategies to handle deadlocks?

A:

1. Deadlock Prevention — Eliminate at least one necessary condition:

- **Eliminate Mutual Exclusion:** Share resources where possible (read-only files). Not always feasible.
- **Eliminate Hold and Wait:** Require processes to request ALL resources at once before starting; or release all held resources before requesting new ones. Low utilization, starvation.
- **Eliminate No Preemption:** If a process can't get a resource, preempt all its held resources. Works for preemptable resources (CPU, memory). Not for printers.
- **Eliminate Circular Wait:** Impose total ordering on resource types; always request in increasing order. Practical and widely used.

2. Deadlock Avoidance — Dynamically ensure system never enters an unsafe state:

- Requires advance knowledge of maximum resource needs.
- **Banker's Algorithm:** Before granting a resource, check if resulting state is "safe" (a safe sequence exists where all processes can finish). If not, make process wait.

3. Deadlock Detection and Recovery:

- Allow deadlocks to occur; periodically detect them; recover.

- Detection: Run cycle detection on RAG (single instance) or resource allocation analysis (multiple instances).
- Recovery:
 - **Process termination:** Abort one or all deadlocked processes.
 - **Resource preemption:** Take resources from deadlocked processes (may require rollback).

4. Deadlock Ignorance (Ostrich Algorithm):

- Ignore the problem; assume deadlocks are rare.
- Used in Windows, Linux (for some cases) — programmer's responsibility.

Q40. What is the Banker's Algorithm?

A: The Banker's Algorithm (Dijkstra) determines if a resource allocation state is **safe** (can all processes eventually complete without deadlock).

Data structures (n processes, m resource types):

- `Available[m]`: Available instances of each resource.
- `Max[n][m]`: Maximum demand of each process.
- `Allocation[n][m]`: Currently allocated to each process.
- `Need[n][m]`: Remaining need = `Max - Allocation`.

Safety Algorithm:

1. `Work = Available, Finish[i] = false` for all i.
2. Find process i where `Finish[i] = false AND Need[i] ≤ Work`.
3. `Work = Work + Allocation[i], Finish[i] = true`. Go to step 2.
4. If `Finish[i] = true` for all i → state is **SAFE**. Otherwise **UNSAFE**.

Resource Request Algorithm: When process P_i requests resources, temporarily allocate them and run safety check. If safe → grant. If unsafe → deny (make P_i wait).

7. Memory Management

Q41. What is memory management? Goals?

A: Memory management is the OS function that manages **allocation, use, and deallocation of main memory (RAM)** among processes.

Goals:

- **Relocation:** Processes can be loaded into any memory location.

- **Protection:** Process cannot access another's memory (hardware enforced).
 - **Sharing:** Allow controlled sharing of memory regions between processes.
 - **Logical organization:** Support modular programming (code, data, stack as separate segments).
 - **Physical organization:** Efficiently move data between RAM and disk.
-

Q42. What is contiguous memory allocation? Types?

A: Each process is stored in a single contiguous block of memory.

Fixed Partitioning:

- Memory divided into fixed-size partitions at system boot.
- Simple, but **internal fragmentation** (allocated partition larger than needed → wasted space inside).
- Limits number of active processes.

Variable Partitioning:

- Partitions created dynamically to exact process size.
- No internal fragmentation.
- **External fragmentation:** Free memory exists but is scattered in non-contiguous holes → can't satisfy large requests even with enough total free space.
- **Solution:** Compaction (move processes to coalesce free space) — expensive!

Allocation Strategies:

- **First Fit:** Allocate first hole that is large enough. Fast.
 - **Best Fit:** Allocate smallest hole that fits. Minimizes wasted space; but leaves tiny unusable fragments.
 - **Worst Fit:** Allocate largest hole. Leaves large remaining fragments; generally worst performance.
-

Q43. What is fragmentation? Internal vs External?

A:

- **Internal Fragmentation:** Wasted space **inside** an allocated block because the block is larger than the request. Common in fixed partitioning and paging (last page of a process may not be fully used).
- **External Fragmentation:** Wasted space **outside** allocated blocks — total free memory is sufficient but split into non-contiguous pieces. Common in variable partitioning and segmentation.

Fragmentation solutions:

- Internal: Smaller allocation units (but more overhead).
 - External: Compaction (expensive), paging/segmentation (non-contiguous allocation).
-

Q44. What is paging in memory management?

A: Paging eliminates external fragmentation by allowing a process's memory to be **non-contiguous** in physical memory.

Key concepts:

- Physical memory divided into fixed-size **frames**.
- Logical memory divided into same-size **pages**.
- OS maintains a **page table** for each process: page number → frame number.
- **Page size:** Power of 2, typically 4 KB.

Address translation:

```
Logical Address = [Page Number (p) | Page Offset (d)]  
Physical Address = [Frame Number (f) | Page Offset (d)]  
where f = PageTable[p]
```

Internal fragmentation in paging: Average 1/2 page wasted per process (last page partially filled).
Smaller pages → less internal fragmentation but larger page table.

Q45. What is segmentation?

A: Segmentation divides a process's logical address space into **variable-size segments** corresponding to logical units (code, data, stack, heap, library).

- Each segment has a **base** (physical start address) and **limit** (size).
- Logical address = <segment number, offset>; offset must be < limit.
- Provides natural protection (each segment has separate R/W/X permissions) and sharing (share code segment).

Problem: External fragmentation (variable-size segments). Solved by combining with paging: **segmented paging** (segment → page table → physical frame). Used in x86 (Intel historically).

8. Virtual Memory

Q46. What is virtual memory and what are its advantages?

A: Virtual memory creates the **illusion that each process has a large, contiguous private address space** — larger than physical RAM — by using disk as an extension of memory.

Advantages:

- Programs can be **larger than physical RAM**.
- More processes can run simultaneously (only active portions need to be in RAM).
- **Memory isolation**: Each process has a private virtual address space.
- **Simplified programming**: No need to manage physical addresses.
- **Efficient sharing**: Same physical frame can be mapped into multiple processes' virtual spaces (e.g., shared libraries).

Implementation: Demand paging — only load pages into memory when they are actually accessed (on demand), not at process start.

Q47. What is demand paging?

A: In demand paging, pages are **loaded into memory only when accessed** (on demand), not preloaded.

Valid/Invalid bit in PTE:

- Valid (1): Page is in memory → access succeeds.
- Invalid (0): Page is NOT in memory → **page fault** triggered.

Page fault handling:

1. CPU accesses a page with invalid bit → page fault exception.
2. OS page fault handler invoked.
3. Check if address is valid (in process's address space). If not → segfault, kill process.
4. Find a free frame; if none, run **page replacement algorithm**.
5. Read the required page from disk (swap space / page file) into the frame. (I/O — high latency!)
6. Update page table entry (set valid bit, fill frame number).
7. Restart the faulting instruction.

Effective Access Time = $(1 - p) \times \text{memory_time} + p \times \text{page_fault_time}$ where p = page fault rate. Even a tiny p causes massive performance impact (disk I/O is 1,000,000× slower than RAM).

Q48. What are page replacement algorithms?

A: When a page fault occurs and no free frames exist, the OS must **evict** a page to make room.

1. Optimal (OPT / Bélády's Algorithm):

- Replace the page that will **not be used for the longest time in the future**.
- Theoretically optimal (minimum page faults) but **impossible to implement** (requires future knowledge).

- Used as a benchmark to evaluate other algorithms.
-

2. FIFO (First-In, First-Out):

- Replace the **oldest** page in memory (the one loaded longest ago).
 - Simple; keep a queue of pages.
 - **Bélády's Anomaly**: Adding more frames can **increase** page faults with FIFO!
 - Not great performance in practice.
-

3. LRU (Least Recently Used):

- Replace the page **not used for the longest time in the past**.
 - Uses temporal locality — recently used pages likely to be used again.
 - Good performance; no Bélády's anomaly.
 - **Implementation**: Counter (timestamp each access, evict smallest) or Stack (move page to top on access, evict bottom).
 - **Problem**: Expensive to implement in hardware exactly. Requires timestamp update on every memory access.
-

4. LRU Approximation — Clock Algorithm (Second Chance):

- Pages arranged in a circle; clock hand sweeps around.
 - Each page has a **reference bit** (set to 1 on access by hardware).
 - When replacement needed: if reference bit = 1 → clear it, advance clock (give second chance). If 0 → replace this page.
 - Good approximation of LRU; efficient implementation.
-

5. MFU (Most Frequently Used):

- Replace the page with the **most accesses** (assumption: was brought in long ago and probably done).
 - Rarely used in practice.
-

6. LFU (Least Frequently Used):

- Replace the page with the **fewest accesses**.
 - Problem: A page used heavily early gets stuck in memory; new pages have low count and get evicted immediately.
-

Q49. What is thrashing?

A: Thrashing is a condition where a process (or the system) **spends more time paging than executing** — constant page faults, swapping pages in and out.

Cause: Process has more active pages (working set) than available frames. OS keeps replacing pages that are immediately needed again.

Effects: CPU utilization drops sharply; I/O (paging disk) becomes saturated; entire system slows to a crawl.

Detection: CPU utilization decreasing while paging I/O increasing.

Solutions:

- **Working Set Model:** Track the set of pages used in the last Δ time units (working set). Ensure each process has enough frames for its working set. If sum of working sets > total frames → suspend some processes.
 - **Page Fault Frequency (PFF):** Monitor page fault rate per process. If too high → allocate more frames. If too low → take frames away.
 - **Reduce multiprogramming:** Swap out processes to give remaining processes enough frames.
-

Q50. What is the working set model?

A: The **working set** $W(t, \Delta)$ of a process at time t is the set of pages referenced in the last Δ time units (the working set window).

- Δ too small → working set too small (won't capture full locality).
- Δ too large → working set too large (old pages retained).
- $\Delta = \infty$ → working set = all pages of the program.

The OS tracks the working set of each process and ensures it has enough frames to hold its working set. If the total working set size > total frames → suspend some processes (reduce multiprogramming level) to prevent thrashing.

9. File Systems

Q51. What is a file system?

A: A file system is the OS component that **manages how data is stored, organized, named, accessed, and protected** on storage devices.

It provides:

- **Naming:** Human-readable names for data.
- **Organization:** Hierarchical directory structure.

- **Access:** Sequential, random, keyed access.
- **Protection:** Permissions and access control.
- **Integrity:** Consistent state after crashes (journaling).
- **Abstraction:** Hides physical storage details.

Q52. What is the structure of a file system?

A:

File System Layers:

Application Programs

↓

Logical File System – File metadata (FCB/inode), directory structure, protection

↓

File-Organization Module – Maps logical blocks to physical blocks; free space management

↓

Basic File System – Issues generic commands to device driver (read block 7, etc.)

↓

I/O Control (Device Driver) – Translates commands to hardware-specific instructions

↓

Devices (Disk, SSD, NVM)

Q53. What is an inode?

A: An **inode (Index Node)** is a data structure (used in Unix/Linux file systems: ext4, XFS) that stores **all metadata about a file** except its name.

Inode contents:

- File type (regular, directory, symlink, device)
- File size
- Owner (UID, GID)
- Permissions (rwxrwxrwx)
- Timestamps: atime (access), mtime (modify), ctime (change)
- Link count (number of hard links)
- **Block pointers:** Direct, single indirect, double indirect, triple indirect (for large files)

Inode

```

├─ Direct pointers (12) → Data blocks
├─ Single Indirect → Block of pointers → Data blocks
├─ Double Indirect → Block → Block of pointers → Data blocks
└─ Triple Indirect → Block → Block → Block of pointers → Data blocks

```

Key: File name is stored in the **directory**, not the inode. A directory maps names → inode numbers.

Q54. What are hard links vs soft (symbolic) links?

A:

Hard Link:

- Multiple directory entries pointing to the **same inode**.
- Deleting one link doesn't delete the file — inode's link count decremented; file deleted only when count = 0.
- Cannot span file systems (inode numbers are FS-specific).
- Cannot link to directories (prevent cycles).

Soft Link (Symbolic Link):

- A special file containing a **path** (string) to another file.
- Deleting the original file → symlink becomes **dangling** (broken).
- Can span file systems.
- Can link to directories.
- Has its own inode.

```
ln file.txt hardlink.txt      # Hard link – same inode
ln -s file.txt softlink.txt   # Symbolic link – different inode, contains path
```

Q55. What are the file allocation methods?

A: How disk blocks are allocated to files:

1. Contiguous Allocation:

- File stored in consecutive disk blocks.
- **Pros:** Fast sequential access (single seek), simple (just base + length), supports direct access.
- **Cons:** External fragmentation; must know file size at creation; file growth difficult.
- Used in: CD-ROMs, DVDs (read-only, known size).

2. Linked Allocation:

- Each block contains data + **pointer to next block**. File = linked list of blocks.
- **Pros:** No external fragmentation; file can grow easily.
- **Cons:** Random access is O(n) (must traverse list); pointer overhead per block; unreliable (broken pointer = lost data).
- **FAT (File Allocation Table):** Variant — keeps all pointers in a separate table (FAT) in memory. Random access faster (scan table, not disk). Used in FAT16, FAT32, exFAT (USB drives, SD cards).

3. Indexed Allocation:

- Each file has an **index block** containing all block pointers.
 - **Pros:** Supports both sequential and random access efficiently; no external fragmentation.
 - **Cons:** Index block overhead; small files waste space in index block.
 - Unix inode uses a multi-level variant (direct + indirect pointers) to handle files of all sizes.
-

Q56. What is directory structure?

A: Directories organize files in a hierarchy.

Single-Level: One directory for all users. Name conflicts. **Two-Level:** Separate directory per user. No sharing. **Tree-Structured:** Hierarchical. Absolute and relative paths. Standard today. **Acyclic Graph:** Allows sharing (hard links, symlinks). Must handle cycles. **General Graph:** Cycles possible. Garbage collection needed.

Directory Entry (ext4): (`inode_number`, `name`, `file_type`). The directory file maps names to inodes.

Q57. What is free space management?

A: The file system must track which disk blocks are free.

Bit Map / Bit Vector:

- 1 bit per block: 0 = free, 1 = allocated.
- Easy to find contiguous free blocks (AND operations).
- Must be kept in memory for efficiency. Size = $\text{disk_blocks} / 8$ bytes.

Linked List of Free Blocks:

- Free blocks linked together; head stored in special location.
- No waste; finding N contiguous blocks requires traversal.

Grouping:

- First free block stores addresses of N free blocks. Last of those N stores addresses of next N, etc.

Counting:

- Store (first free block, count) pairs. Good for contiguous free space.
-

Q58. What is journaling in file systems?

A: Journaling ensures **file system consistency after a crash** by recording changes to a circular log (journal) before applying them to the file system.

Write-ahead logging for file systems:

1. Write intent (journal entry) to the journal (circular log on disk).

2. Commit the journal entry.
3. Apply changes to the actual file system structures (checkpoint).
4. Remove the journal entry.

On crash: At recovery, replay uncommitted journal entries → file system is consistent (may lose last write, but never corrupted).

Journaling modes (ext4):

- **writeback:** Only metadata journaled. Fastest. May have data corruption.
- **ordered** (default): Metadata journaled; data written before metadata commit. Good balance.
- **journal:** Both data and metadata journaled. Safest but slowest (double writes).

Examples: ext3, ext4, NTFS, APFS, XFS, btrfs all use journaling.

Q59. What are common file systems?

A:

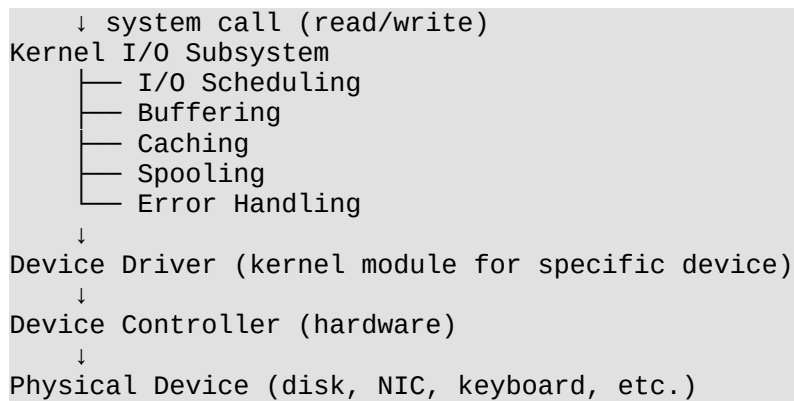
File System	OS	Features
FAT32/exFAT	Windows, cross-platform	Simple; USB drives; max file 4GB (FAT32)
NTFS	Windows	Journaling, ACLs, compression, encryption
ext4	Linux	Journaling, extents, large files, stable
XFS	Linux	High performance, large files, parallel I/O
btrfs	Linux	Copy-on-write, snapshots, RAID, checksums
APFS	macOS, iOS	Copy-on-write, snapshots, encryption, SSD-optimized
ZFS	Solaris, FreeBSD, Linux	Copy-on-write, checksums, RAID-Z, snapshots
HFS+	macOS (legacy)	Case-insensitive, journaling

10. I/O Systems & Device Management

Q60. What is the I/O subsystem architecture?

A:

User Process



Q61. What is buffering, caching, and spooling?

A:

Buffering:

- Temporary storage in memory while data is transferred between device and process.
- Purposes: Handle speed mismatch (fast CPU, slow disk); handle size mismatch (network packets vs application reads); enable copy semantics (app can modify buffer after write call).
- Types: Single buffering, double buffering (producer fills one buffer while consumer drains the other).

Caching:

- Keeping a **copy of frequently accessed data** in faster storage (memory) so subsequent accesses don't need to go to the slower device.
- Key difference from buffering: A buffer holds the **only** copy of data in transit; a cache holds a **copy** of data that also exists elsewhere.
- **Page cache** in Linux: All file I/O goes through the page cache (kernel memory).

Spooling (Simultaneous Peripheral Operations On-Line):

- A buffer for devices that cannot accept **interleaved** data streams (e.g., printer).
- Each job's output held in a spool file; OS sends jobs to printer one at a time.
- Allows multiple processes to "print" simultaneously without interfering.

Q62. What are disk scheduling algorithms?

A: Disk scheduling decides the **order to service disk I/O requests** to minimize seek time (moving the read/write head).

Seek Time: Time to move arm to the cylinder. Dominant cost.

1. FCFS (First-Come, First-Served):

- Service requests in arrival order.
 - Fair but inefficient — head moves back and forth wildly.
-

2. SSTF (Shortest Seek Time First):

- Service the request **closest to current head position** first.
 - Reduces total seek time; better than FCFS.
 - **Starvation**: Requests far from current position may wait forever.
-

3. SCAN (Elevator Algorithm):

- Head sweeps back and forth across disk, servicing requests in the current direction.
 - At the end of the disk, reverses direction.
 - No starvation; uniform service time across disk.
-

4. C-SCAN (Circular SCAN):

- Like SCAN but head only services requests on the way in one direction.
 - After reaching end, jumps back to beginning (without servicing on return trip).
 - More uniform wait times than SCAN.
-

5. LOOK / C-LOOK:

- Like SCAN/C-SCAN but head only goes as far as the last request in each direction (doesn't go to the extreme end).
 - More efficient than SCAN/C-SCAN.
 - **C-LOOK**: Most commonly used in practice.
-

11. Storage Management

Q63. What is RAID? Explain RAID levels.

A: RAID (Redundant Array of Independent Disks) uses multiple disks to improve **performance, reliability (redundancy), or both**.

RAID Level	Description	Min Disks	Redundancy	Use Case
RAID 0 (Striping)	Data striped across disks. No redundancy.	2	None	Max performance; video editing

RAID Level	Description	Min Disks	Redundancy	Use Case
RAID 1 (Mirroring)	Data duplicated on 2 disks.	2	Full (survives 1 disk loss)	Critical data, OS drives
RAID 5 (Striping + Parity)	Data + parity distributed across all disks.	3	Survives 1 disk loss	General storage, NAS
RAID 6 (Dual Parity)	Two parity blocks distributed.	4	Survives 2 disk losses	High reliability storage
RAID 10 (1+0)	Mirror of stripes.	4	Survives multiple losses (one per mirror)	High performance + reliability

RAID 5 write penalty: Every write requires read-modify-write for parity → expensive for write-heavy workloads.

Q64. What is the difference between HDD and SSD in context of OS storage management?

A:

Aspect	HDD	SSD
Storage medium	Magnetic spinning platters	NAND flash memory
Seek time	5–10 ms (mechanical)	Near-zero (no mechanical parts)
Random access	Slow (seeks)	Very fast
Sequential access	Good	Very fast
Scheduling	Disk scheduling algorithms matter	Less critical (near-zero seek)
Wear	Unlimited (in practice)	Write cycles limited; wear leveling needed
I/O scheduler	CFQ, Deadline	NOOP / none (managed by SSD controller)

FTL (Flash Translation Layer): SSD firmware handles wear leveling, garbage collection, bad block management — presents a block device interface to the OS.

12. Security & Protection

Q65. What is the difference between protection and security in OS?

A:

- **Protection:** Mechanisms to control access of processes and users to resources **within the system** — ensuring authorized access only. Focuses on internal system mechanisms (access

control matrix, capabilities).

- **Security:** Defending the system against **external threats** — unauthorized access, malware, denial of service, data theft. Broader scope including authentication, encryption, auditing.

Protection is a prerequisite for security.

Q66. What is access control? Access Control Matrix?

A: Access control determines **which subjects (users/processes) can perform which operations on which objects (files, devices, memory)**.

Access Control Matrix: A 2D table where rows = subjects, columns = objects, cells = permitted operations.

	File1	File2	Printer
Alice	{r, w}	{r}	{print}
Bob	{r}	{r, w}	{ }
Root	{r, w, x}	{r, w, x}	{print}

Implementations:

- **Access Control Lists (ACL):** Column of the matrix stored with each object. "File1 can be read by Alice, Bob." `ls -l` in Linux shows simplified ACL (owner, group, others).
 - **Capability List:** Row of the matrix stored with each subject. "Alice has read-write on File1, print on Printer." Harder to revoke.
-

Q67. What is authentication in OS?

A: Authentication verifies the identity of users trying to access the system.

Methods:

- **Something you know:** Password, PIN, security question.
- **Something you have:** Smart card, hardware token, phone (TOTP).
- **Something you are:** Biometrics (fingerprint, face, iris).
- **Multi-Factor Authentication (MFA):** Combines two or more factors.

Password storage: Passwords never stored in plaintext. Stored as salted hashes (bcrypt, scrypt, argon2).

Unix password hashing: `/etc/shadow` stores `$algorithm$salt$hash`.

Q68. What are common OS security threats?

A:

- **Malware:** Viruses (attach to files), worms (self-replicate via network), trojans (disguised as legitimate), ransomware (encrypt files for ransom), rootkits (hide in OS).

- **Privilege Escalation:** Exploiting bugs to gain higher privileges (root/kernel).
- **Buffer Overflow:** Write past buffer boundary, overwrite return address → execute attacker code. Mitigated by: ASLR, stack canaries, NX bit (non-executable stack), PIE.
- **SQL Injection, XSS:** Application-level (web apps).
- **Denial of Service (DoS):** Exhaust system resources.
- **Man-in-the-Middle:** Intercept communication.
- **Social Engineering / Phishing:** Trick users into revealing credentials.
- **Side-channel attacks:** Spectre, Meltdown — extract data via CPU timing/cache behavior.

OS defenses: ASLR (Address Space Layout Randomization), DEP/NX (no-execute), Secure Boot, SELinux/AppArmor (mandatory access control), sandboxing, code signing.

13. Virtualization & Containers

Q69. What is virtualization?

A: Virtualization creates **virtual versions of hardware resources** — allowing multiple OS instances to run on one physical machine.

Types:

- **Full Virtualization:** Guest OS runs unmodified; VMM translates all hardware access. Example: VMware ESXi, VirtualBox.
 - **Paravirtualization:** Guest OS modified to use VMM APIs directly (hypercalls) instead of hardware traps. Faster than full virtualization. Example: Xen (older).
 - **Hardware-assisted Virtualization:** CPU provides virtualization extensions (Intel VT-x, AMD-V). Guest OS runs at near-native speed; traps to VMM only when necessary. Most modern approach.
 - **OS-Level Virtualization (Containers):** Share the host OS kernel; isolate processes using namespaces and cgroups. Lightweight.
-

Q70. What is a Hypervisor (VMM)?

A: A **hypervisor (Virtual Machine Monitor)** is software/firmware that creates and manages virtual machines.

Type 1 (Bare-metal): Runs directly on hardware; no host OS. Most efficient for servers.

- Examples: VMware ESXi, Microsoft Hyper-V, Xen, KVM (Linux kernel module).

Type 2 (Hosted): Runs as an application on top of a host OS.

- Examples: VMware Workstation, VirtualBox, Parallels.

Type 1 is preferred for production (lower overhead, more secure). Type 2 is convenient for development/testing.

Q71. What are containers? How do they differ from VMs?

A: Containers are **isolated processes** that share the host OS kernel but have their own filesystem, network, and resource limits.

Aspect	Virtual Machine	Container
Isolation	Full OS isolation	Process-level isolation (shared kernel)
OS	Separate OS per VM	Shared host kernel
Size	GBs (full OS)	MBs (just app + deps)
Startup	Minutes	Seconds
Overhead	High (hypervisor + guest OS)	Very low
Security	Strong isolation	Weaker (shared kernel; escape risk)
Use case	Different OS, strong isolation	Microservices, cloud-native apps
Examples	VMware, VirtualBox, KVM	Docker, Podman, LXC

Container technologies:

- **Namespaces:** Isolate PID, network, mount, user, IPC, UTS namespaces.
 - **cgroups (Control Groups):** Limit CPU, memory, I/O, network resources per container.
 - **Union File System (OverlayFS):** Layered file system for container images.
-

14. Distributed Operating Systems

Q72. What is a distributed system?

A: A distributed system is a collection of **independent computers that appears to users as a single coherent system**. Nodes communicate over a network.

Goals: Resource sharing, transparency (location, access, failure), openness, scalability, fault tolerance.

Challenges: Network partitions, node failures, latency, clock synchronization, consistency, security.

Q73. What is clock synchronization in distributed systems?

A: Each node has an independent physical clock that drifts over time. Coordinated time is essential for ordering events.

Network Time Protocol (NTP): Synchronizes clocks over a network to within milliseconds of UTC. Hierarchical (strata).

Logical Clocks (Lamport Timestamps):

- Each process maintains a counter C .
- Before sending a message: increment C , include in message.
- On receiving a message: $C = \max(C_{\text{local}}, C_{\text{msg}}) + 1$.
- Establishes **happened-before** ordering. If $a \rightarrow b$ then $C(a) < C(b)$. But $C(a) < C(b)$ does NOT imply $a \rightarrow b$.

Vector Clocks:

- Each process maintains a vector of counters (one per process).
 - Captures causality precisely. Can determine if two events are concurrent.
-

Q74. What is a distributed file system?

A: A DFS allows files to be stored across multiple servers but accessed transparently as if local.

- **NFS (Network File System):** Client-server. Files stored on server, accessed over network. Stateless protocol (NFSv3), stateful (NFSv4).
 - **HDFS (Hadoop Distributed File System):** Designed for huge files; write-once-read-many; replicates blocks ($3\times$ default) for fault tolerance. Namenode (metadata) + Datanodes (data).
 - **GFS (Google File System):** Similar to HDFS; chunk servers + master; handles failures as the norm.
 - **Ceph:** Distributed object store + file system; no single point of failure; self-healing.
-

15. Linux Internals & System Calls

Q75. What happens when you type `ls` in a Linux terminal?

A:

1. Shell reads the command "`ls`".
2. Shell calls `fork()` → creates child process.
3. Child calls `exec("/bin/ls")` → loads `ls` binary into memory.
4. `ls` calls `getdents64()` system call → kernel reads directory entries from file system.
5. `ls` calls `write()` to stdout (file descriptor 1) → kernel sends data to terminal.

6. `ls` calls `exit()`.
7. Kernel sends `SIGCHLD` to parent shell.
8. Shell calls `wait()` to collect child's exit status.
9. Shell displays next prompt.

Q76. What is the Linux process hierarchy?

A:

```
PID 0 – swapper (idle process, runs when nothing else runs)
PID 1 – init / systemd (ancestor of all user processes)
├── sshd (SSH daemon)
│   └── bash (user shell after login)
│       └── vim (editor started by user)
├── cron (job scheduler)
├── nginx (web server)
└── ... other daemons
```

Orphan adoption: When a parent dies before its child, the child is adopted by **PID 1** (`init/systemd`).

Q77. What are Linux signals?

A: Signals are **asynchronous notifications** sent to processes.

Signal	Number	Default Action	Cause
SIGHUP	1	Terminate	Terminal hangup; daemon reload
SIGINT	2	Terminate	Ctrl+C
SIGQUIT	3	Core dump	Ctrl+\
SIGKILL	9	Terminate	Cannot be caught or ignored
SIGSEGV	11	Core dump	Segmentation fault (invalid memory access)
SIGTERM	15	Terminate	Graceful termination request
SIGCHLD	17	Ignore	Child stopped or terminated
SIGSTOP	19	Stop	Cannot be caught; Ctrl+Z effect
SIGCONT	18	Continue	Resume stopped process
SIGUSR1/2	10/12	Terminate	User-defined signals

Processes can install **signal handlers** for most signals. SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

Q78. What is the /proc filesystem in Linux?

A: /proc is a **virtual filesystem** (no real disk storage) that exposes kernel and process information as files. It's a window into the kernel's internals.

/proc/cpuinfo	– CPU model, cores, flags
/proc/meminfo	– RAM usage, swap, buffers
/proc/version	– Kernel version
/proc/uptime	– System uptime
/proc/loadavg	– Load averages (1, 5, 15 min)
/proc/<PID>/	– Information about process PID
/proc/<PID>/status	– Process state, memory, UID, GID
/proc/<PID>/cmdline	– Command line arguments
/proc/<PID>/fd/	– File descriptors (symlinks)
/proc/<PID>/maps	– Memory mappings
/proc/<PID>/environ	– Environment variables

16. Advanced OS Concepts

Q79. What is the difference between preemptive and cooperative multitasking?

A:

- **Cooperative Multitasking:** Each process voluntarily yields the CPU when it's done or waiting. OS cannot forcibly preempt. A misbehaving or stuck process can freeze the system. Used in early Windows (3.x), classic Mac OS.
 - **Preemptive Multitasking:** OS uses a timer interrupt to forcibly take CPU from processes after their time quantum. No process can monopolize the CPU. Used in all modern OS (Linux, Windows NT+, macOS).
-

Q80. What is a real-time operating system (RTOS)?

A: An RTOS guarantees that tasks are completed within **strict timing deadlines**.

Hard Real-Time: Missing a deadline = system failure. Life-critical. Examples: pacemakers, airbag controllers, aircraft fly-by-wire.

Soft Real-Time: Missing a deadline = degraded performance, not failure. Examples: video streaming, online gaming, VoIP.

Key characteristics: Deterministic scheduling, minimal interrupt latency, priority-based preemption, no unbounded blocking.

Scheduling in RTOS:

- **Rate-Monotonic Scheduling (RMS):** Assign priority based on period (shorter period =

higher priority). Optimal for fixed-priority preemptive scheduling.

- **Earliest Deadline First (EDF):** Dynamic priority = earliest absolute deadline. Optimal for preemptive scheduling.

Examples: FreeRTOS, VxWorks, QNX, RTLinux.

Q81. What is memory-mapped I/O vs regular I/O?

A:

- **Regular I/O (Port I/O):** Use special CPU instructions (IN/OUT in x86) to communicate with device registers through a separate I/O address space.
- **Memory-Mapped I/O (MMIO):** Device registers mapped into the regular memory address space. CPU uses standard `load/store` instructions to read/write device registers.

MMIO advantages: No special instructions; works with C pointers; can use memory protection; more portable (ARM has no separate I/O space).

mmap() system call: Maps a file or device directly into a process's virtual address space. File I/O becomes memory reads/writes — can be faster than `read( )/write( )` for large files (no kernel buffer copy).

Q82. What is the difference between a monolithic kernel and microkernel in terms of performance and reliability?

A:

Monolithic (e.g., Linux):

- Device drivers, file systems, schedulers all run in kernel space.
- Fast: Function calls within kernel space (no context switch or IPC for services).
- Reliability risk: A buggy driver crashes the kernel (Blue Screen of Death, kernel panic).
- Linux mitigates via loadable kernel modules (LKM) — dynamic loading without rebooting.

Microkernel (e.g., QNX, MINIX 3):

- Only IPC, memory management, scheduling in kernel.
- Device drivers, file systems run in user space.
- Slower: Each kernel service requires message passing (IPC) — multiple context switches.
- More reliable: A crashed driver can be restarted without kernel crash.
- More secure: Smaller kernel = smaller attack surface (lower TCB — Trusted Computing Base).

Modern trend: Hybrid kernels (Windows NT, macOS XNU) and driver isolation techniques attempt to combine best of both.

Q83. What is the difference between `malloc()` and `brk()/mmap()` in Linux?

A:

- **`brk()/sbrk()`**: Extends or shrinks the process heap by moving the "program break" (end of data segment). Used by `malloc` for small allocations. Simple but not thread-safe for large allocations.
- **`mmap()`**: Maps a new anonymous memory region anywhere in the virtual address space. Used by `malloc` for large allocations (> 128KB typically). Can be independently `munmap()`'d. Thread-safe.

`malloc()` implementation (glibc):

- Small allocations (< 128KB): Use `brk()`; manage in a free list (bins by size class).
 - Large allocations (≥ 128KB): Use `mmap()`; each allocation is its own mapping.
 - `free()` for `mmap`'d memory calls `munmap()`; for `brk` memory, returns to free list.
-

Q84. What are kernel threads vs user threads in Linux?

A: In Linux, **every thread is a kernel thread** (Linux's `task_struct` represents both processes and threads). Linux uses `clone()` system call with flags to create threads vs processes:

- `fork()` internally calls `clone()` with minimal sharing.
- `pthread_create()` calls `clone()` with `CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND` — sharing address space, file system, file descriptors, signal handlers.

Each thread has its own `task_struct` and is independently scheduled by the Linux scheduler. POSIX threads (pthreads) are implemented as kernel threads in Linux (1:1 model).

Q85. What is copy-on-write (COW)?

A: Copy-on-write is an optimization where when a resource is "copied" (e.g., `fork()`), both copies initially **share the same physical pages** — no actual copying. When either process **writes** to a shared page, only then is a private copy made for that process.

`fork()` with COW:

1. Child and parent share all physical pages (marked read-only).
2. On first write by either process → page fault → OS creates a private copy for the writing process → write proceeds.
3. Pages never written are never copied → very cheap `fork()` when followed by `exec()`.

COW is also used in: btrfs/ZFS snapshots, database buffer management, string handling in some languages (Python 2 strings), Redis persistence (BGSAVE).

Q86. What is NUMA (Non-Uniform Memory Access)?

A: In multi-socket servers, each CPU socket has **local memory** it can access quickly and **remote memory** (on other sockets) it accesses more slowly via an interconnect (QPI/HyperTransport).

```
Socket 0: CPU0 — Local RAM 0 (fast)
              \
                — Remote RAM 1 (slow – via interconnect)
Socket 1: CPU1 — Local RAM 1 (fast)
```

OS NUMA awareness:

- **NUMA-aware memory allocation:** Allocate memory on the same node as the CPU running the thread.
- **CPU affinity:** Bind threads to specific cores/sockets.
- Linux: `numactl`, `/proc/sys/kernel/numa_balancing` for automatic NUMA balancing.
- Memory access to the wrong NUMA node can be 2-3× slower.

Q87. What is the difference between hard and soft interrupts?

A:

- **Hard Interrupt (Hardware IRQ):** Triggered directly by hardware (keyboard, disk, NIC). High priority; CPU immediately suspends current task and runs the interrupt handler (ISR). ISR must be fast — often just acknowledges device and schedules work.
- **Soft Interrupt / Bottom Half / Tasklet / Workqueue:** The "deferred" part of interrupt handling. The top half (ISR) does minimal work and schedules the bottom half to run later (still in kernel context, but lower priority). Examples: Linux `softirqs`, tasklets, workqueues.

Linux interrupt handling split:

- **Top half (hardirq):** Fast, runs with interrupts disabled. Acknowledge device, save data.
- **Bottom half (softirq/tasklet):** Slower processing (TCP/IP stack, disk I/O completion) with interrupts re-enabled.

Q88. What is the OOM Killer in Linux?

A: When the system is critically low on memory and cannot satisfy an allocation (even after swapping), the Linux **OOM (Out-of-Memory) Killer** is invoked.

It selects a process to terminate based on an **OOM score** (computed from: memory usage, CPU time, priority, whether process is privileged, etc.).

```
cat /proc/<PID>/oom_score          # Current OOM score (higher = more likely to be
killed)
echo -1000 > /proc/<PID>/oom_score_adj # -1000: never kill; +1000: always kill
first
```

OOM kills are logged in `dmesg`: "Out of memory: Kill process <PID>".

Q89. What is **strace** and **ltrace**?

A:

- **strace**: Traces all **system calls** made by a process. Invaluable for debugging — shows what kernel interactions (file opens, network calls, memory maps) a program makes.

```
strace ls -l          # Trace system calls made by ls
strace -p 1234        # Attach to running process
strace -e trace=read,write ls # Filter to specific syscalls
```

- **ltrace**: Traces **library function calls** (calls to shared libraries like libc). Shows `malloc()`, `printf()`, `strlen()` etc.

Both are essential Linux debugging tools.

Q90. What is the difference between **exec**, **spawn**, and **fork** in process creation?

A:

- **fork()**: Creates a **copy** of the current process. Parent and child share code; child is a clone. Copy-on-write for efficiency. Returns twice (once in parent, once in child). Unix/Linux.
- **exec()**: **Replaces** the current process's image with a new program. Same PID, new code/data/stack. Used after `fork()` to run a different program in the child.
- **spawn()**: Creates a **new process to run a specific program** directly — combines `fork+exec` in one step. More efficient (no unnecessary COW pages). Used in Windows (`CreateProcess`), POSIX `posix_spawn()`, Python `subprocess.Popen`.

The Unix "fork-exec" pattern vs Windows "spawn" pattern is a fundamental OS design difference.

❑ Quick Reference Cheat Sheet

OS Fundamentals

Concept	Key Point
Kernel types	Monolithic (Linux), Microkernel (QNX), Hybrid (Windows)
Kernel vs User mode	Ring 0 (all privileges) vs Ring 3 (restricted)
System call	Controlled crossing from user → kernel mode
Process vs Thread	Separate address space vs shared address space
PCB	Kernel data structure storing all process info

Process States & Management

Concept	Key Point
Process states	New → Ready → Running → Blocked → Terminated
Context switch	Save/restore PCB; pure overhead
fork()	Creates child process (copy-on-write)
exec()	Replaces process image
Zombie	Finished but parent hasn't called wait()
Orphan	Parent died; adopted by PID 1 (init)
IPC methods	Pipe, FIFO, Shared Memory, Message Queue, Signal, Socket

CPU Scheduling

Algorithm	Key Feature	Problem
FCFS	Arrival order	Convoy effect
SJF	Shortest burst first	Starvation, needs prediction
SRTF	Preemptive SJF	Starvation
Round Robin	Time quantum, fair	Quantum size tuning
Priority	Priority-based	Starvation → aging
MLFQ	Adaptive; I/O-bound promoted	Complexity
CFS (Linux)	vruntime red-black tree	—

Synchronization

Concept	Key Point
Critical section	Mutual exclusion + Progress + Bounded waiting
Mutex	Ownership; blocking; only owner unlocks
Semaphore	Counter; wait/signal; no ownership
Monitor	High-level; implicit mutex + condition vars
Spinlock	Busy-wait; good for short CS on multicore
Peterson's	Software solution for 2 processes
Producer-Consumer	3 semaphores: mutex, empty, full
Readers-Writers	Multiple readers OR one writer

Deadlocks

Concept	Key Point
Necessary conditions	Mutual exclusion + Hold&Wait + No preemption + Circular wait
Prevention	Eliminate one condition
Avoidance	Banker's algorithm; safe state check
Detection	RAG cycle; recover by termination/preemption
Ostrich	Ignore (Linux/Windows for some cases)

Memory Management

Concept	Key Point
Paging	Fixed-size pages/frames; eliminates external fragmentation
Segmentation	Variable-size; logical units; external fragmentation
Internal fragmentation	Wasted space inside allocated block
External fragmentation	Wasted space between blocks
TLB	Cache for page table entries; >99% hit rate
Demand paging	Load pages only when accessed
Page replacement	OPT (optimal) > LRU > Clock $\approx$ LFU > FIFO
Thrashing	More paging than computing; fix with working set
COW	Fork shares pages until write; then copy

File Systems

Concept	Key Point
Inode	All file metadata except name; block pointers
Hard link	Multiple names $\rightarrow$ same inode
Symlink	File containing path string
Contiguous	Fast sequential; external fragmentation
Linked/FAT	No fragmentation; no random access
Indexed/inode	Best of both; multi-level for large files
Journaling	WAL for FS consistency after crash
ext4	Linux standard; ordered journaling default

Deadlock & Scheduling Quick Math

Formula	Meaning
Turnaround = Completion – Arrival	Total time in system
Waiting = Turnaround – Burst	Time spent in ready queue
Response = First CPU – Arrival	Time to first response
Effective Access Time = $(1-p) \times T_{\text{mem}} + p \times T_{\text{pagefault}}$	Virtual memory performance
CPU Utilization $\approx 1 - (I/O \text{ fraction})^n$	Multiprogramming benefit (n processes)

Master these concepts and you'll be fully prepared for any Operating Systems interview!